Marena — Cloud & DevOps Architect

(Interview Preparation)

> **Role:** Cloud & DevOps Architect | **Company:** Mareana
> **Experience:** 10-12 years | **Location:** Bangalore (Full-Time)
> **Primary Cloud:** AWS (with Azure/GCP reference knowledge)
> **Focus Areas:** Cloud Architecture, DevOps, Kubernetes (EKS), Terraform, CI/CD, GitOps, Networking, Security
> **Company Domain:** Enterprise AI/ML products for Manufacturing, Sustainability & Supply Chain
> **Prepared by:** Pushparaj Naik

About Mareana

Mareana builds **Enterprise-Scale AI/ML-powered products** for:

- **Manufacturing**: Smart factory, process optimization, quality control
- **Sustainability**: Carbon tracking, ESG compliance, resource optimization
- **Supply Chain**: Demand forecasting, logistics optimization, inventory management

This role builds the **AWS platform layer** — designing, securing, and automating the cloud infrastructure that powers Mareana's AI/ML products. It's a platform engineering leadership role.

Table of Contents

---

1. AWS Cloud Architecture — Core

Q1: Walk me through designing a multi-account AWS architecture for an enterprise SaaS product.

Answer:

I design enterprise AWS environments using **AWS Organizations + Control Tower** with a clear account separation strategy:

```
AWS ORGANIZATIONS — ACCOUNT STRUCTURE:

Root (Management Account)
├── Security OU
│   ├── Log Archive Account
│   │   └── Centralized: CloudTrail, VPC Flow Logs, Config snapshots
│   ├── Security Tooling Account
│   │   └── Delegated admin: GuardDuty, Security Hub, Inspector, Macie
│   └── Audit Account
│       └── AWS Config aggregator, cross-account read-only access
│
├── Infrastructure OU
│   ├── Network Hub Account
│   │   └── Transit Gateway, Direct Connect, Route 53 zones, NAT
│   └── Shared Services Account
│       └── CI/CD (Jenkins), ECR, Artifact storage, AMI pipeline
│
├── Workloads OU
│   ├── Dev Account      (relaxed SCPs, small instances)
│   ├── Staging Account  (prod-mirror, stricter SCPs)
│   └── Production Account (full SCPs, HA, DR)
│
```

```
└── Sandbox OU
    └── Experimentation (auto-expire resources after 30 days)
```

Governance enforcement:

Control	Mechanism	Example
Preventive	SCPs (Service Control Policies)	Deny root user access, restrict regions to us-east-1 + us-west-2
Detective	AWS Config Rules	Detect unencrypted S3 buckets, public security groups
Proactive	CloudFormation Guard / Terraform policies	Block non-compliant Terraform before apply

Key design decisions:

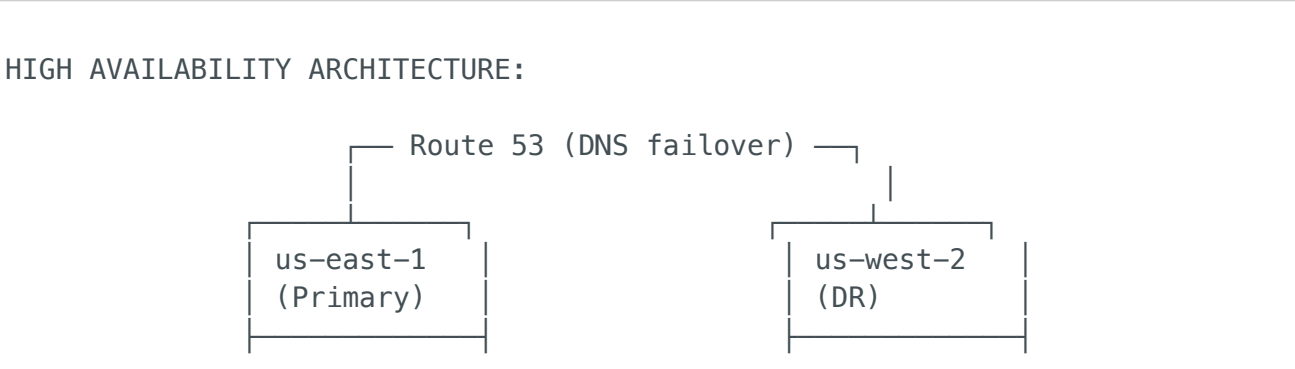
- **IAM Identity Center (SSO)**: Federated with corporate IdP (Okta/Entra ID), no IAM users
- **Transit Gateway**: Hub-and-spoke networking across accounts
- **Centralized logging**: All CloudTrail + VPC Flow Logs → Log Archive (S3 + KMS)
- **Terraform**: Remote state in S3 + DynamoDB locking, one state per account per component
- **Account vending**: Control Tower Account Factory + Terraform for standardized provisioning

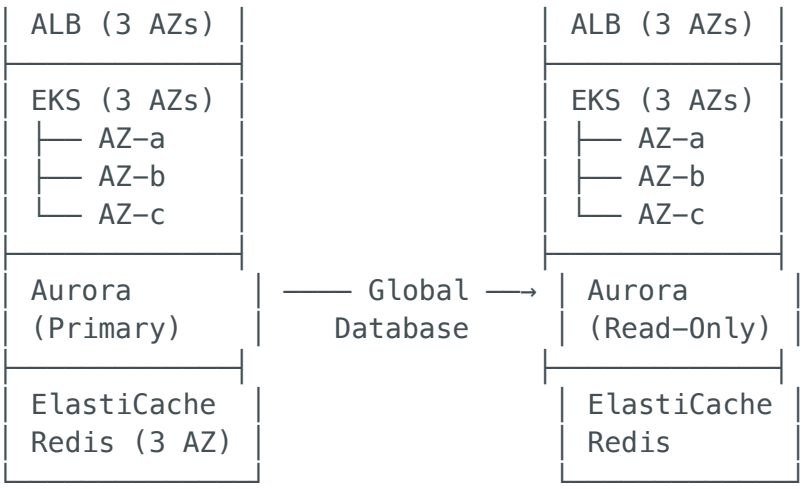
---

Q2: How do you design a highly available, fault-tolerant architecture on AWS?

Answer:

HA on AWS means **eliminating single points of failure at every layer:**





Layer	HA Strategy	RPO	RTO
DNS	Route 53 health checks + failover routing	0	< 60s
CDN	CloudFront (global edge)	N/A	0
Load Balancer	ALB across 3 AZs	N/A	0
Compute (EKS)	Multi-AZ node groups + PodDisruptionBudgets	N/A	< 30s
Database	Aurora Multi-AZ (synchronous) + Global Database (async)	0 (local), <1s (cross-region)	< 30s (local), < 1min (cross-region)
Cache	ElastiCache Redis Multi-AZ with auto-failover	0	< 30s
Storage	S3 (11 9's durability) + Cross-Region Replication	< 15min	< 1hr
Messaging	SQS (distributed, no single AZ dependency)	0	0

DR testing I run quarterly:

1. Simulate primary region failure (Route 53 failover)
2. Promote Aurora read-replica in DR region
3. Run automated integration tests against DR endpoints
4. Measure actual RTO vs target — document gaps
5. Share findings in post-mortem format

Q3: Explain the AWS Well-Architected Framework and how you apply each pillar.

Answer:

Pillar	My Practical Application	Key AWS Tools
Operational Excellence	100% IaC (Terraform), GitOps (ArgoCD), automated runbooks, post-mortem culture	CloudFormation, Systems Manager, CloudWatch
Security	Zero-trust VPC, KMS CMK encryption, least-privilege IAM, DevSecOps pipeline	GuardDuty, Security Hub, IAM, KMS, Macie
Reliability	Multi-AZ everything, auto-scaling (Karpenter), chaos engineering (FIS), tested DR	Auto Scaling, Route 53, FIS, Aurora Global
Performance Efficiency	Right-sizing (Compute Optimizer), caching (ElastiCache), async queues (SQS)	Compute Optimizer, ElastiCache, SQS, Lambda
Cost Optimization	Savings Plans, Spot for EKS (Karpenter), S3 lifecycle, Kubecost dashboards	Cost Explorer, Budgets, Savings Plans, Spot
Sustainability	Graviton (ARM) instances, serverless for event-driven, right-sized resources	Graviton, Lambda, Karpenter

I run **quarterly Well-Architected Reviews** using the AWS Well-Architected Tool, prioritize findings by risk, and track remediation in Jira sprints.

Q4: Describe key AWS compute services and when to use each.

Answer:

Service	Use Case	When to Use	When NOT to Use
EC2	General compute, full OS control	Legacy apps, GPU workloads, custom AMIs	Stateless microservices (use containers)
EKS	Container orchestration	Microservices at scale, complex networking, multi-team	Simple single-container apps
ECS Fargate	Serverless containers	Simple services, no K8s expertise needed	Complex service mesh, multi-cloud portability
Lambda	Event-driven compute	API handlers, S3 triggers, cron jobs, glue logic	Long-running tasks (>15min), sustained load
Auto Scaling Groups	Scaling EC2 fleets	VM-based apps, stateful workloads	Container workloads (use K8s HPA/Karpenter)

For Mareana's AI/ML platform, my compute strategy:

- **EKS**: Application microservices + ML inference endpoints
- **Lambda**: Event processing (S3 triggers, IoT events), API Gateway backends
- **EC2 (GPU)**: ML training workloads (p4d/g5 instances via EKS GPU node groups)
- **Fargate**: Batch jobs, CI runners (no node management overhead)

Q5: Explain AWS storage services — S3, EBS, EFS. When do you use each?

Answer:

Service	Type	Performance	Durability	Use Case
S3	Object storage	Scales infinitely, 3,500 PUT/s per prefix	99.999999999% (11 9's)	ML datasets, logs, backups, static assets, Terraform state

Service	Type	Performance	Durability	Use Case
EBS gp3	Block storage	3,000 IOPS (baseline), 125 MB/s	99.8-99.9%	EC2 root volumes, database storage
EBS io2	Block storage	Up to 64,000 IOPS	99.999%	High-performance databases (RDS, self-managed)
EFS	Network file system	Elastic throughput	99.999999999%	Shared storage across multiple pods/EC2, CMS content
FSx for Lustre	High-performance parallel	100+ GB/s throughput	Built on S3	ML training (read large datasets fast)

S3 lifecycle policy I implement:

```
Day 0–30:   S3 Standard (frequent access)
Day 31–90:  S3 Intelligent-Tiering (auto-optimizes)
Day 91–365: S3 Glacier Instant Retrieval
Day 366+:   S3 Glacier Deep Archive
```

For Mareana: S3 for ML data lake (sensor data, model artifacts), EBS gp3 for RDS, EFS for shared config/models across EKS pods.

Q6: Explain AWS database services — RDS, Aurora, DynamoDB, ElastiCache, Redshift.

Answer:

Service	Type	Best For	HA	Scaling
RDS PostgreSQL	Relational	Standard OLTP, familiar SQL	Multi-AZ	Read replicas (up to 15)

Service	Type	Best For	HA	Scaling
Aurora PostgreSQL	Relational (cloud-native)	High-performance OLTP, global apps	Multi-AZ + Global DB	Auto-scaling read replicas, serverless
DynamoDB	Key-value / Document	High-throughput metadata, session state, IoT telemetry	Multi-AZ (built-in)	On-demand or provisioned, global tables
ElastiCache Redis	In-memory cache	Session cache, API response cache, leaderboards	Multi-AZ, auto-failover	Cluster mode sharding
Redshift	Data warehouse	Analytics, BI reporting, large aggregations	Multi-AZ (RA3)	Serverless or provisioned
OpenSearch	Search & analytics	Full-text search, log analytics, vector search	Multi-AZ	Instance scaling

For Mareana's platform:

- **Aurora PostgreSQL**: Primary transactional database (OLTP — orders, configurations, metadata)
- **DynamoDB**: IoT sensor metadata, session state, high-throughput key-value lookups
- **ElastiCache Redis**: API response caching, feature store for ML inference
- **Redshift Serverless**: Supply chain analytics, BI dashboards
- **OpenSearch**: Log analysis (ELK alternative), manufacturing process search

---

2. AWS Networking — VPC, Hybrid, DNS

Q7: Design a production VPC architecture for Mareana.

Answer:

```
PRODUCTION VPC ARCHITECTURE:

VPC: 10.0.0.0/16 (65,536 IPs)
Region: us-east-1
AZs: us-east-1a, us-east-1b, us-east-1c

┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  PUBLIC SUBNETS (one per AZ):                                    │
│  ├── 10.0.1.0/24 (AZ-a) ─── NAT Gateway + ALB                   │
│  ├── 10.0.2.0/24 (AZ-b) ─── NAT Gateway + ALB                   │
│  └── 10.0.3.0/24 (AZ-c) ─── NAT Gateway + ALB                   │
│                                                                  │
│  PRIVATE SUBNETS — APPLICATION (one per AZ):                     │
│  ├── 10.0.11.0/24 (AZ-a) ─── EKS worker nodes                   │
│  ├── 10.0.12.0/24 (AZ-b) ─── EKS worker nodes                   │
│  └── 10.0.13.0/24 (AZ-c) ─── EKS worker nodes                   │
│                                                                  │
│  PRIVATE SUBNETS — DATA (one per AZ):                            │
│  ├── 10.0.21.0/24 (AZ-a) ─── RDS, ElastiCache                   │
│  ├── 10.0.22.0/24 (AZ-b) ─── RDS, ElastiCache                   │
│  └── 10.0.23.0/24 (AZ-c) ─── RDS, ElastiCache                   │
│                                                                  │
│  SECURITY:                                                       │
│  ├── Security Groups (stateful):                                 │
│  │   ├── sg-alb: 443 from 0.0.0.0/0                             │
│  │   ├── sg-eks-nodes: All from sg-alb, all from self            │
│  │   ├── sg-rds: 5432 from sg-eks-nodes only                     │
│  │   └── sg-redis: 6379 from sg-eks-nodes only                   │
│  ├── NACLs (stateless): Deny rules for defense-in-depth          │
│  └── Flow Logs → CloudWatch Logs + S3 (KMS encrypted)            │
│                                                                  │
│  VPC ENDPOINTS (PrivateLink — no internet traversal):            │
│  ├── Gateway: S3, DynamoDB (free)                                │
│  ├── Interface: ECR (dkr + api), STS, Secrets Manager, KMS       │
│  ├── Interface: CloudWatch Logs, SSM, SQS, SNS                   │
│  └── Interface: EKS API (private endpoint)                       │
│                                                                  │
│  DNS:                                                            │
│  ├── VPC DNS resolution: Enabled                                 │
│  ├── VPC DNS hostnames: Enabled                                  │
│  ├── Route 53 Private Hosted Zone: mareana.internal              │
│  └── Route 53 Resolver: Forward rules for on-prem DNS            │
└──────────────────────────────────────────────────────────────────┘
```

Q8: How do you implement hybrid cloud connectivity on AWS?

Answer:

```
HYBRID CONNECTIVITY:

On-Premises (Manufacturing Facility / Corporate DC)
      │
      ├── AWS Direct Connect (10 Gbps) ──── PRIMARY PATH
      │   └── Dedicated private connection, low latency (~2ms)
      │
      ├── Site-to-Site VPN (IPSec) ──── BACKUP PATH
      │   └── Over internet, encrypted, BGP routing
      │
      └── Both terminate at:
            │
         Transit Gateway (hub)
            │
            ├──────┬──────┬──────────┐
            │      │      │          │
       Prod VPC  Dev VPC  Shared    Network
                          Services  Hub VPC

TRANSIT GATEWAY DESIGN:
  ├── Route table per environment (prod → prod only, dev → dev only)
  ├── Cross-account sharing via RAM (Resource Access Manager)
  ├── Blackhole routes to prevent dev → prod traffic
  └── Flow logs for cross-VPC traffic auditing

DNS (Hybrid Resolution):
  ├── AWS → On-prem: Route 53 Resolver outbound endpoint
  │   └── Forward *.corp.company.com → on-prem DNS servers
  ├── On-prem → AWS: Route 53 Resolver inbound endpoint
  │   └── Forward *.mareana.internal → Route 53 private zones
  └── Split-horizon DNS for same domain names

FIREWALL (Network Firewall):
  ├── AWS Network Firewall in network hub VPC
  ├── Stateful rules: IDS/IPS (Suricata-compatible)
  ├── Domain allow-listing for internet egress
  └── TLS inspection for outbound traffic
```

Q9: What are VPC Endpoints and why are they critical for security?

Answer:

VPC Endpoints allow resources in private subnets to access AWS services **without traversing the internet or NAT Gateway**:

Endpoint Type	Services	Cost	Use Case
Gateway	S3, DynamoDB	Free	Always use — saves NAT Gateway data charges
Interface (PrivateLink)	ECR, STS, Secrets Manager, KMS, CloudWatch, SQS, SSM, etc.	$0.01/hr + data	Private access to AWS APIs from EKS/EC2

Why critical:

1. **Security**: Traffic stays within AWS network — no internet exposure
2. **Cost**: Avoids NAT Gateway data processing charges ($0.045/GB)
3. **Compliance**: Required for private EKS clusters (no public API endpoint)
4. **Performance**: Lower latency (no NAT hop)

For EKS private clusters, I always provision these endpoints:

```
Required VPC Endpoints for Private EKS:
├── com.amazonaws.region.eks                 (EKS API)
├── com.amazonaws.region.ecr.api             (ECR — pull images)
├── com.amazonaws.region.ecr.dkr             (ECR — Docker registry)
├── com.amazonaws.region.s3                  (S3 — ECR image layers)
├── com.amazonaws.region.sts                 (STS — IRSA token exchange)
├── com.amazonaws.region.logs                (CloudWatch Logs)
├── com.amazonaws.region.elasticloadbalancing (ALB provisioning)
└── com.amazonaws.region.secretsmanager      (Secrets)
```

Endpoint policies — I attach restrictive policies:

```
{
  "Statement": [{
    "Effect": "Allow",
    "Principal": "*",
    "Action": "ecr:*",
    "Resource": "arn:aws:ecr:us-east-1:ACCOUNT_ID:repository/mareana-*",
    "Condition": {
      "StringEquals": { "aws:PrincipalOrgID": "o-xxxxxxxxxx" }
    }
  }]
}
```

Q10: How do you troubleshoot DNS issues in AWS?

Answer:

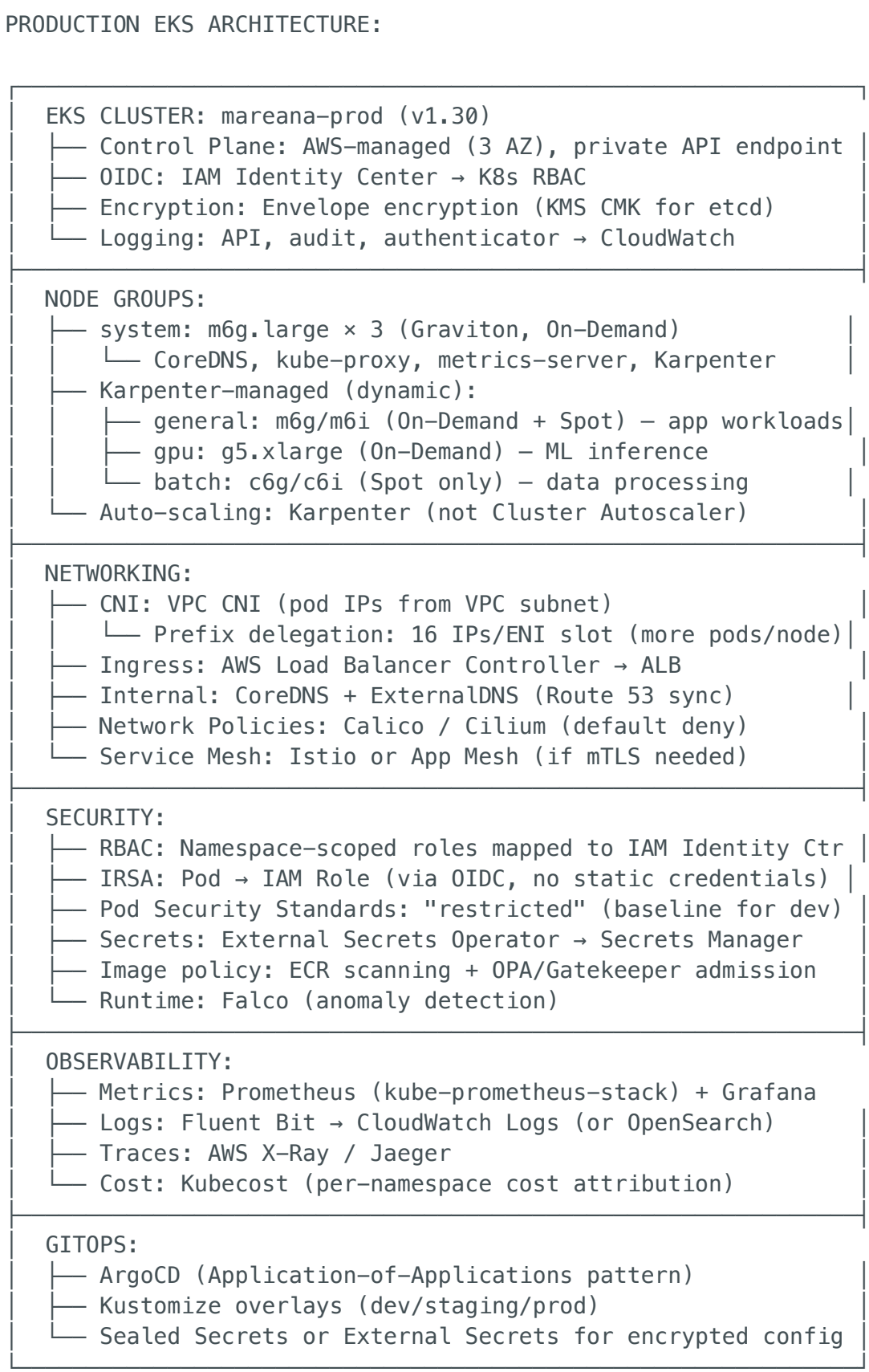
```
DNS TROUBLESHOOTING METHODOLOGY:

Step 1: IDENTIFY SCOPE
├── Is it all DNS or a specific record?
├── Is it from a pod (CoreDNS), EC2 (VPC DNS), or hybrid (Resolver)?
└── dig / nslookup from the failing source

Step 2: KUBERNETES DNS (CoreDNS)
├── kubectl logs -n kube-system -l k8s-app=kube-dns
├── kubectl exec -it <pod> -- nslookup <service>.default.svc.cluster.local
├── Common issues:
│   ├── ndots: 5 (default) — causes 4 extra DNS queries per lookup
│   │   Fix: Set ndots: 2 in pod spec dnsConfig
│   ├── CoreDNS OOMKilled — increase memory limits
│   ├── Search domain too long — fails silently
│   └── CoreDNS cache poisoning — upgrade + verify CoreDNS ConfigMap
└── Check: kubectl get cm coredns -n kube-system -o yaml

Step 3: VPC DNS
├── dig @169.254.169.253 <hostname> (VPC resolver)
├── Check: VPC enableDnsSupport = true
├── Check: VPC enableDnsHostnames = true
├── Check: DHCP options set → DNS servers correct
└── Check: Route 53 Private Hosted Zone associated with VPC

Step 4: HYBRID DNS (Route 53 Resolver)
├── Check: Resolver rules for on-prem domains
├── Check: Security groups on resolver endpoints (port 53 TCP+UDP)
├── Check: Resolver endpoint ENIs in correct subnets
└── Test: dig @<resolver-inbound-ip> <on-prem-hostname>

Step 5: TOOLS
├── dig +trace <domain>          (follow full delegation chain)
├── tcpdump -i any port 53       (capture DNS packets)
├── nslookup -type=ANY <domain>  (query all record types)
├── aws route53 test-dns-answer  (test Route 53 resolution)
└── CloudWatch Logs → Route 53 Query Logging
```

---

3. Kubernetes on AWS — EKS at Scale

Q11: How do you design and manage production EKS clusters?

Answer:



Q12: What is Karpenter and how does it replace Cluster Autoscaler on EKS?

Answer:

Feature	Cluster Autoscaler	Karpenter
Scaling speed	2-5 minutes (waits for pending pods → ASG scale)	30-60 seconds (calls EC2 API directly)
Instance selection	Fixed per node group (e.g., m5.xlarge only)	Dynamic — evaluates 100+ instance types per pod request
Spot handling	1 instance type per ASG (poor diversification)	Automatic diversification across instance families
Consolidation	None (never shrinks underutilized nodes)	Built-in — bin-packs pods, terminates underutilized nodes
ARM/Graviton	Separate node group required	`kubernetes.io/arch: arm64` constraint — auto-selects
GPU	Separate node group	Auto-selects GPU instance when pod requests `nvidia.com/gpu`

My Karpenter configuration:

```yaml
apiVersion: karpenter.sh/v1beta1
kind: NodePool
metadata:
  name: default
spec:
  template:
    spec:
      nodeClassRef:
        name: default
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["on-demand", "spot"]      # Spot for cost savings
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64", "arm64"]          # Graviton where possible
        - key: karpenter.k8s.aws/instance-category
          operator: In
          values: ["m", "c", "r"]             # General/compute/memory
```

```
        - key: karpenter.k8s.aws/instance-generation
          operator: Gt
          values: ["5"]                          # Gen 6+ only
  disruption:
    consolidationPolicy: WhenUnderutilized     # Auto-shrink
    expireAfter: 720h                           # Recycle every 30 days
  limits:
    cpu: "500"
    memory: 1000Gi
```

Result: Karpenter reduced my EKS infrastructure costs by ~35% through:

- Automatic Spot diversification (60-70% cheaper nodes)
- Graviton (ARM) selection (20% cheaper)
- Consolidation (terminate underutilized nodes)
- Right-sizing (launch exact instance type for workload)

Q13: How do you perform zero-downtime EKS cluster upgrades?

Answer:

```
EKS UPGRADE PROCESS (e.g., 1.29 → 1.30):

Week -1: PREPARATION
├── Review K8s 1.30 changelog for API deprecations
├── Run: kubectl deprecations (pluto or kubent tool)
├── Test upgrade on dev cluster first
├── Verify add-on compatibility matrix:
│   ├── VPC CNI → check compatible version
│   ├── CoreDNS → check compatible version
│   ├── kube-proxy → check compatible version
│   └── Karpenter → check compatible version
├── Verify Helm chart compatibility (ingress, cert-manager, ArgoCD)
└── Update PodDisruptionBudgets (ensure minAvailable ≥ 1)

Day 1: CONTROL PLANE UPGRADE
├── aws eks update-cluster-version --name prod --kubernetes-version 1.30
├── AWS rolls API servers across 3 AZs (zero pod disruption)
├── Duration: ~25-40 minutes
├── Validate: kubectl get nodes (nodes still on 1.29 — expected)
└── Validate: API server responds correctly

Day 1: MANAGED ADD-ONS UPGRADE
├── aws eks update-addon --name vpc-cni --addon-version v1.18.x
├── aws eks update-addon --name coredns --addon-version v1.11.x
├── aws eks update-addon --name kube-proxy --addon-version v1.30.x
```

```
└── Validate: kubectl get pods -n kube-system (all healthy)

Day 2: NODE ROTATION
├── Option A (Karpenter): Update NodePool → set expireAfter: 0h
│   └── Karpenter drains old nodes, launches new 1.30 nodes automatically
├── Option B (Managed NG): Create new node group (1.30) → drain old → delete
├── PodDisruptionBudgets ensure graceful migration
└── Validate: kubectl get nodes (all nodes on 1.30)

Day 2: HELM RELEASES
├── Update Helm releases that need K8s 1.30 compatibility
├── ArgoCD auto-syncs updated manifests
└── Validate: All workloads healthy

Day 3: CLEANUP
├── Delete old node group (if not Karpenter)
├── Update Terraform to reflect new K8s version
├── Document upgrade in runbook
└── Keep rollback plan ready for 48 hours
```

Q14: How do you handle Kubernetes security hardening on EKS?

Answer:

```
EKS SECURITY — DEFENSE IN DEPTH:

Layer 1: CLUSTER
├── Private API endpoint (no public access)
├── Envelope encryption for etcd (KMS CMK)
├── Audit logging → CloudWatch Logs (all API calls)
├── IMDSv2 enforced on all nodes (block metadata v1)
└── OIDC provider for IRSA (pod-level IAM)

Layer 2: NODES
├── Managed node groups (Amazon Linux 2023, auto-patched)
├── CIS Benchmark validated (kube-bench)
├── No SSH access — use SSM Session Manager
├── Node security group: only allow from control plane + ALB
└── Karpenter: AMI auto-updates on node rotation

Layer 3: NAMESPACES
├── RBAC:
│   ├── dev-team → Role (pods, services, configmaps in ns-dev)
│   ├── platform-team → ClusterRole (all namespaces, limited verbs)
│   └── ci-bot → Role (deployments/update only)
├── ResourceQuotas: CPU/memory limits per namespace
├── LimitRanges: Default requests/limits for all pods
└── NetworkPolicies: default-deny ingress/egress + allow-list
```

```
Layer 4: PODS
├── Pod Security Standards ("restricted" profile):
│   ├── runAsNonRoot: true
│   ├── readOnlyRootFilesystem: true
│   ├── allowPrivilegeEscalation: false
│   ├── seccompProfile: RuntimeDefault
│   └── capabilities: drop: ["ALL"]
├── External Secrets Operator (no K8s Secrets in Git)
├── IRSA (every pod → specific IAM Role, no node-level access)
└── Service account token auto-mount disabled (unless needed)

Layer 5: IMAGES
├── ECR image scanning (Enhanced — Inspector integration)
├── Admission controller: OPA/Gatekeeper
│   ├── Only allow images from approved ECR repos
│   ├── Block latest tag
│   ├── Require image digest (SHA)
│   └── Block hostNetwork, hostPID, privileged
├── Image signing: cosign (Sigstore)
└── Base images: Distroless or Amazon-managed minimal

Layer 6: RUNTIME
├── Falco: Real-time threat detection
│   ├── Detect: shell in container, unexpected network, file write
│   ├── Alert: CloudWatch → SNS → PagerDuty
│   └── Respond: Quarantine pod (remove from Service)
└── Network policies: Cilium (eBPF-based, higher performance)
```

Q15: Explain IRSA (IAM Roles for Service Accounts) on EKS.

Answer:

IRSA allows Kubernetes pods to assume IAM roles **without static credentials**:

```
HOW IRSA WORKS:

1. EKS cluster has an OIDC provider (identity issuer)
2. IAM Role trusts the OIDC provider (trust policy)
3. K8s ServiceAccount is annotated with the IAM Role ARN
4. Pod using that ServiceAccount gets temporary STS credentials
5. AWS SDK in the pod automatically uses these credentials

FLOW:
  Pod (ServiceAccount: s3-reader)
    ↓ mount projected token (JWT)
  AWS STS (AssumeRoleWithWebIdentity)
    ↓ validate JWT with OIDC provider
  IAM Role (arn:aws:iam::123456:role/s3-reader-role)
```

```
    ↓ return temporary credentials
  Pod → S3 API (with scoped credentials)
```

Terraform setup:

```
# IAM Role for specific K8s ServiceAccount
module "irsa_s3_reader" {
  source  = "terraform-aws-modules/iam/aws//modules/iam-role-for-service-
accounts-eks"

  role_name = "mareana-s3-reader"

  oidc_providers = {
    main = {
      provider_arn = module.eks.oidc_provider_arn
      namespace_service_accounts = ["production:s3-reader"]
    }
  }

  role_policy_arns = {
    s3_read = aws_iam_policy.s3_read_only.arn
  }
}
```

Why IRSA is critical:

- **No static credentials**: No AWS_ACCESS_KEY_ID in pods or environment variables
- **Least privilege**: Each service gets exactly the permissions it needs
- **Audit**: CloudTrail shows which pod assumed which role
- **Auto-rotation**: STS credentials expire in 1 hour (configurable)

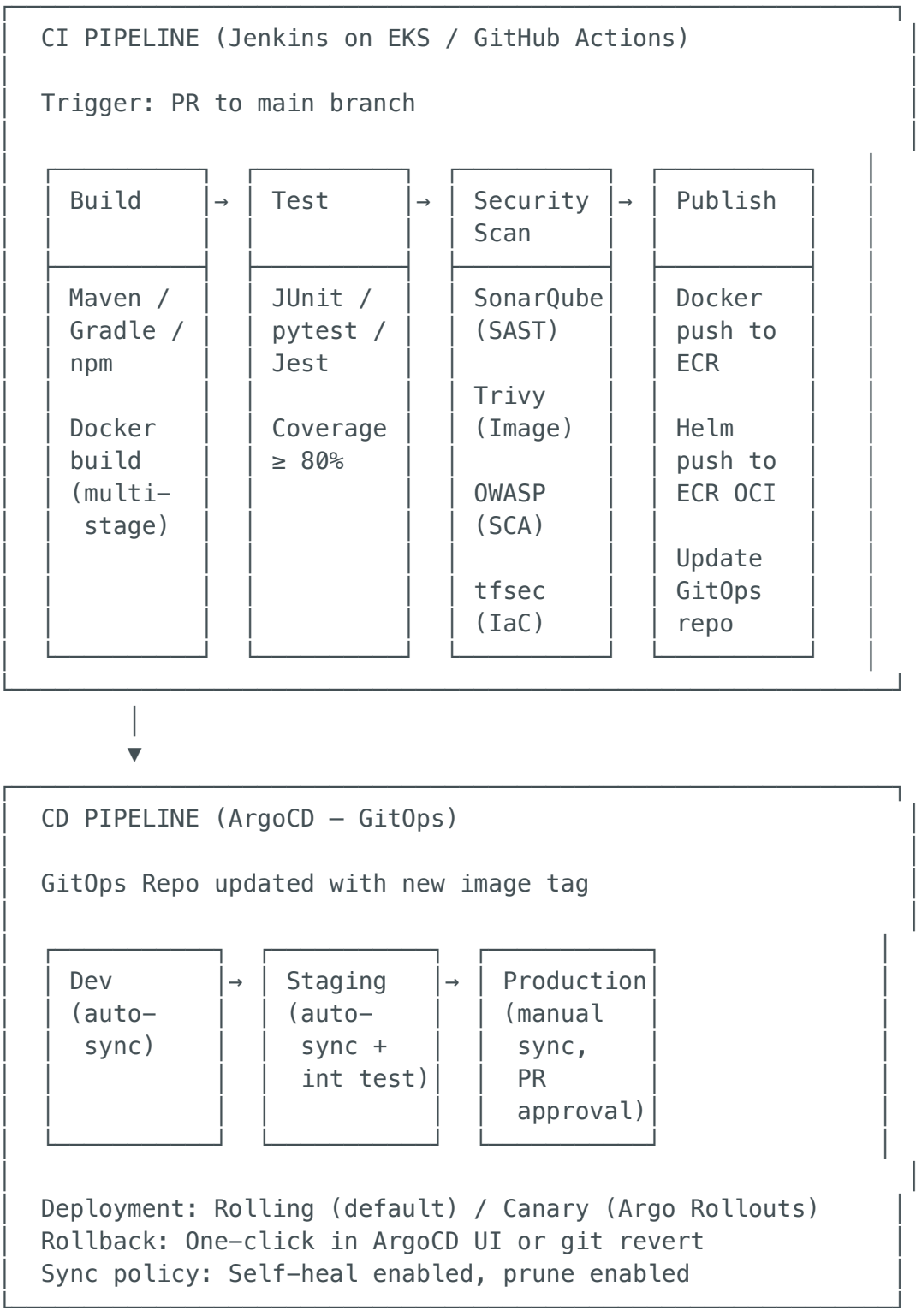
---

4. CI/CD Pipelines & GitOps

Q16: Design a production CI/CD pipeline on AWS.

Answer:

```
CI/CD ARCHITECTURE:
```



Q17: How do you implement GitOps with ArgoCD on EKS?

Answer:

```
ARGOCD ON EKS:

Installation:
```

```
  helm install argocd argo/argo-cd -n argocd \
    --set server.ingress.enabled=true \
    --set server.ingress.ingressClassName=alb \
    --set configs.params.server.insecure=true  # TLS at ALB

Repository structure (Kustomize-based):
  gitops-repo/
  ├── apps/                          # Application manifests
  │   ├── storefront-api/
  │   │   ├── base/
  │   │   │   ├── deployment.yaml
  │   │   │   ├── service.yaml
  │   │   │   ├── hpa.yaml
  │   │   │   └── kustomization.yaml
  │   │   └── overlays/
  │   │       ├── dev/               # replicas: 1, resources: small
  │   │       ├── staging/           # replicas: 2, resources: medium
  │   │       └── prod/              # replicas: 3, PDB, anti-affinity
  │   └── order-service/
  │       └── ... (same structure)
  │
  ├── infrastructure/                # Platform components
  │   ├── cert-manager/
  │   ├── external-secrets/
  │   ├── ingress-nginx/
  │   ├── kube-prometheus-stack/
  │   └── karpenter/
  │
  └── root-app.yaml                  # App-of-Apps (bootstraps everything)

ArgoCD Application manifest:
  apiVersion: argoproj.io/v1alpha1
  kind: Application
  metadata:
    name: storefront-api-prod
    namespace: argocd
  spec:
    project: production
    source:
      repoURL: https://github.com/mareana/gitops.git
      targetRevision: main
      path: apps/storefront-api/overlays/prod
    destination:
      server: https://kubernetes.default.svc
      namespace: production
    syncPolicy:
      automated:
        selfHeal: true    # Fix manual drift
        prune: true       # Remove deleted resources
      syncOptions:
        - CreateNamespace=true
```

ArgoCD RBAC (multi-team):

```
ArgoCD Projects:
├── production:  Only platform-admin can sync
├── staging:     Team leads can sync
├── dev:         Any developer can sync
└── infrastructure: Only platform-admin
```

Q18: Compare Jenkins, GitHub Actions, GitLab CI/CD for this role.

Answer:

Feature	Jenkins	GitHub Actions	GitLab CI/CD
Hosting	Self-managed (EKS/EC2)	GitHub SaaS	Self-managed or SaaS
Pipeline DSL	Groovy (Jenkinsfile)	YAML (workflow)	YAML (.gitlab-ci.yml)
Plugins	1800+ (largest ecosystem)	Actions marketplace (growing)	Built-in features
K8s runners	K8s plugin (dynamic agents on EKS)	Self-hosted runners on EKS	K8s executor (native)
Secrets	Credentials + Vault plugin	Repository/Org secrets	CI/CD Variables
ECR integration	Docker pipeline plugin	aws-actions/amazon-ecr-login	Built-in Docker support
Best for	Complex pipelines, enterprise legacy, maximum customization	GitHub-native teams, modern workflows	Full DevSecOps platform (SCM+CI+CD+Registry)

My recommendation for Mareana:

- **Jenkins on EKS** (if existing investment) — maximum flexibility, dynamic K8s agents
- **GitHub Actions** (if new setup) — simpler, SaaS, great AWS integrations
- **ArgoCD for CD** (regardless of CI choice) — separate CI from CD for security

---

5. Infrastructure as Code — Terraform on AWS

Q19: How do you structure Terraform for multi-environment AWS infrastructure?

Answer:

```
TERRAFORM STRUCTURE:

terraform/
├── modules/                          # Reusable, versioned modules
│   ├── networking/
│   │   ├── main.tf                   # VPC, subnets, NAT, TGW
│   │   ├── variables.tf              # CIDR, AZs, VPC endpoint config
│   │   ├── outputs.tf                # VPC ID, subnet IDs
│   │   └── versions.tf               # Provider version constraints
│   ├── eks/
│   │   ├── main.tf                   # EKS cluster, OIDC, add-ons
│   │   ├── karpenter.tf              # Karpenter provisioner
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── rds/
│   │   ├── main.tf                   # Aurora cluster, subnet group
│   │   └── variables.tf
│   ├── elasticache/
│   ├── monitoring/
│   └── security/
│       ├── main.tf                   # IAM roles, KMS keys, Config rules
│       └── variables.tf
│
├── environments/
│   ├── dev/
│   │   ├── main.tf                   # module "vpc" { source =
"../../modules/networking" }
│   │   ├── terraform.tfvars          # eks_node_min=2,
rds_instance_class=db.t3.medium
│   │   └── backend.hcl              # bucket = "mareana-tf-state-dev"
│   ├── staging/
│   │   ├── main.tf
│   │   ├── terraform.tfvars          # eks_node_min=3,
```

```
rds_instance_class=db.r6g.large
│   │   └── backend.hcl
│   └── prod/
│       ├── main.tf
│       ├── terraform.tfvars         # eks_node_min=6,
rds_instance_class=db.r6g.xlarge
│       └── backend.hcl
│
└── .github/workflows/
    └── terraform.yml                 # PR: plan + tfsec; Merge: apply
```

Terraform CI pipeline:

```
PR Created:
  ├── terraform init → terraform fmt -check → terraform validate
  ├── tfsec (security scan — HIGH/CRITICAL fail the PR)
  ├── checkov (compliance — CIS benchmarks)
  ├── terraform plan → output posted as PR comment
  ├── infracost diff → cost estimate posted as PR comment
  └── Reviewer approves (code + plan + cost reviewed)

PR Merged:
  ├── terraform plan (re-validate)
  ├── Manual approval gate (for staging/prod)
  ├── terraform apply -auto-approve
  └── Slack notification (success/failure)
```

Q20: How do you handle Terraform state management and common pitfalls?

Answer:

```
STATE MANAGEMENT:

Backend:
  S3 (state files) + DynamoDB (state locking)
  ├── S3: Versioned, SSE-KMS encrypted, MFA-delete
  ├── DynamoDB: Pay-per-request, LockID partition key
  ├── IAM: Env-specific roles (dev role can't access prod state)
  └── Lifecycle: Never manual edits to state — always via Terraform CLI
```

Pitfall	Solution
State corruption	S3 versioning → roll back; always `terraform plan` before `apply`
Lock contention	DynamoDB lock table; `terraform force-unlock` as last resort
Drift	Nightly `terraform plan` → CloudWatch alarm if changes detected
Secrets in state	SSE-KMS encryption; never store secrets in `.tf` files
Large state	Split into logical domains (networking, EKS, databases, monitoring)
Refactoring	`terraform state mv` for renames; `terraform import` for existing resources
Cross-env deps	`terraform_remote_state` data source or SSM Parameter Store
Provider updates	Pin provider versions; test upgrades in dev first

---

6. Docker & Container Best Practices

Q21: How do you build secure, optimized Docker images?

Answer:

```
# PRODUCTION DOCKERFILE — Multi-stage, hardened
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21-alpine AS builder
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline -B         # Layer cache: dependencies
COPY src/ src/
RUN mvn clean package -DskipTests -B

# Stage 2: Runtime (minimal attack surface)
FROM eclipse-temurin:21-jre-alpine
```

```
RUN addgroup -S app && adduser -S -G app app \
    && apk add --no-cache tini                # PID 1 signal handling
WORKDIR /app
COPY --from=builder --chown=app:app /build/target/*.jar app.jar
USER app
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD wget -qO- http://localhost:8080/health/live || exit 1
ENTRYPOINT ["tini", "--"]
CMD ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0", \
     "-XX:+UseG1GC", "-jar", "app.jar"]
```

Security checklist:

Practice	Why	How
Multi-stage build	Reduce attack surface (no build tools in prod)	`FROM ... AS builder`
Non-root user	Prevent privilege escalation	`USER app`
Read-only root filesystem	Prevent file system writes	K8s `readOnlyRootFilesystem: true`
No `latest` tag	Reproducibility	Use git SHA or semantic version
`.dockerignore`	Don't copy secrets, .git, node_modules into image	Explicit allow-list
Minimal base	Fewer CVEs	Alpine, Distroless, or scratch
HEALTHCHECK	Container orchestrator knows if app is healthy	`HEALTHCHECK CMD`
tini as PID 1	Proper signal handling (SIGTERM)	`ENTRYPOINT ["tini", "--"]`

7. Monitoring, Observability & Logging on AWS

Q22: Design an observability stack for EKS workloads on AWS.

Answer:

```
THREE PILLARS ON AWS:

┌──────────────────────────────────────────────────────────────┐
│  METRICS: Prometheus + Grafana                               │
│  ├── Deploy: kube-prometheus-stack (Helm)                    │
│  ├── Or: Amazon Managed Prometheus (AMP) + Managed Grafana   │
│  ├── Service discovery: ServiceMonitor CRDs                  │
│  ├── Retention: 15 days local; Thanos for long-term (S3)     │
│  ├── Alerting: AlertManager → PagerDuty (P1/P2), Slack (P3)  │
│  └── Key dashboards:                                         │
│      ├── Cluster: CPU, memory, pods, nodes                   │
│      ├── Application: Request rate, error rate, latency (RED)│
│      ├── Business: Transactions/sec, active users, queue     │
│      └── Cost: Kubecost per-namespace spend                  │
├──────────────────────────────────────────────────────────────┤
│  LOGS: Fluent Bit → CloudWatch Logs (or OpenSearch)          │
│  ├── Fluent Bit: DaemonSet on every node                     │
│  │   ├── Container stdout/stderr → structured JSON           │
│  │   ├── Kubernetes metadata enrichment (pod, namespace)     │
│  │   └── Multi-output: CloudWatch Logs + S3 (archive)        │
│  ├── CloudWatch Logs Insights: Ad-hoc query language         │
│  ├── Or OpenSearch: Kibana dashboards, full-text search      │
│  ├── Retention: 30 days hot, 90 days warm, 365 days S3       │
│  └── Also: CloudTrail logs, VPC Flow Logs, ALB access logs   │
├──────────────────────────────────────────────────────────────┤
│  TRACES: AWS X-Ray or Jaeger                                 │
│  ├── Instrumentation: OpenTelemetry SDK (auto-instrumentation)│
│  ├── Collector: ADOT (AWS Distro for OpenTelemetry) sidecar  │
│  ├── Backend: X-Ray (managed) or Jaeger (self-hosted on EKS) │
│  └── Value: Trace requests across microservices for latency  │
└──────────────────────────────────────────────────────────────┘

ALERTING HIERARCHY:
  P1 (SEV1): Production down     → PagerDuty call    → 5 min response
  P2 (SEV2): Degraded service    → PagerDuty + Slack  → 15 min response
  P3 (SEV3): Non-prod issue      → Slack only         → 1 hour response
  P4 (SEV4): Informational       → Dashboard only     → Next business day

KEY PROMETHEUS ALERTS I ALWAYS CONFIGURE:
  ├── KubePodCrashLooping (restart > 3 in 10min)
  ├── KubeDeploymentReplicasMismatch (desired ≠ ready)
  ├── HighErrorRate (5xx > 5% of total for 5min)
  ├── HighLatency (p99 > 2s for 5min)
  ├── NodeNotReady (node condition != Ready for 5min)
  ├── PVCAlmostFull (usage > 85%)
  ├── CertificateExpiringSoon (< 30 days)
  └── KubeQuotaExceeded (namespace hitting resource limits)
```

---

8. AWS Security, IAM & DevSecOps

Q23: How do you implement DevSecOps on AWS?

Answer:

```
DEVSECOPS — SHIFT LEFT:

CODE PHASE:
├── Pre-commit hooks: gitleaks (secrets), hadolint (Dockerfile)
├── IDE: SonarLint (real-time code quality)
└── Branch protection: PR required, signed commits, 2 reviewers

BUILD PHASE (CI):
├── SAST: SonarQube (code vulnerabilities + code smells)
├── SCA: OWASP Dependency Check (library CVEs)
├── Container: Trivy (CRITICAL/HIGH = fail pipeline)
├── IaC: tfsec + checkov (Terraform security)
├── Secrets: truffleHog (detect committed credentials)
└── SBOM: Syft (software bill of materials)

DEPLOY PHASE (CD):
├── Image signing: cosign / Sigstore
├── OPA/Gatekeeper: Admission policies (block non-compliant pods)
├── RBAC: Namespace-scoped permissions
├── Network policies: Default deny + allow-list
└── Secrets: External Secrets Operator → AWS Secrets Manager

RUNTIME (Production):
├── AWS WAF: OWASP Top 10 managed rules on ALB
├── GuardDuty: Threat detection (malicious IPs, crypto mining, DNS exfil)
├── Security Hub: Aggregated compliance view (CIS, PCI, AWS Best Practices)
├── Inspector: Continuous EC2/ECR vulnerability scanning
├── Falco: K8s runtime anomaly detection
├── Macie: S3 PII discovery and classification
└── CloudTrail: API audit logging (all accounts → Log Archive)

GOVERNANCE:
├── AWS Config: 50+ managed rules for continuous compliance
├── SCPs: Preventive guardrails across accounts
├── IAM Access Analyzer: Find unused permissions, public access
└── Quarterly penetration testing
```

Q24: How do you design IAM for an AWS organization?

Answer:

```
IAM STRATEGY:

1. HUMAN ACCESS:
   ├── IAM Identity Center (SSO)
   │   ├── Federate with corporate IdP (Okta / Entra ID)
   │   ├── Permission Sets → AWS accounts
   │   │   ├── AdministratorAccess → Prod (Platform admins only)
   │   │   ├── PowerUserAccess → Dev (Dev team)
   │   │   ├── ReadOnlyAccess → All (Everyone)
   │   │   └── CustomDevOps → Staging (CI/CD team)
   │   ├── Session duration: 1 hour (prod), 4 hours (dev)
   │   └── MFA enforced on all users
   ├── Zero IAM users (all access via SSO)
   └── Zero long-lived access keys

2. SERVICE ACCESS:
   ├── EC2/ECS: Instance profiles with scoped IAM roles
   ├── EKS pods: IRSA (ServiceAccount → IAM Role via OIDC)
   ├── Lambda: Execution role (least-privilege per function)
   ├── CI/CD: OIDC federation (GitHub Actions → IAM Role, no secrets)
   └── Cross-account: Role assumption with ExternalId condition

3. LEAST PRIVILEGE ENFORCEMENT:
   ├── IAM Access Analyzer (generate least-privilege policies from
CloudTrail)
   ├── Permission boundaries (cap maximum permissions for delegated admin)
   ├── SCPs (deny actions at account level — overrides IAM allow)
   ├── S3 bucket policies + VPC endpoint policies
   └── Resource-based policies (KMS key policies, SQS policies)

4. KEY POLICIES (SCPs I always deploy):
   ├── deny root user (except for break-glass)
   ├── deny creating IAM users/access keys
   ├── deny disabling CloudTrail
   ├── deny creating public S3 buckets
   ├── deny leaving approved regions
   └── deny launching unapproved instance types (in sandbox)
```

---

9. Cloud Migration & Legacy Modernization

Q25: How do you approach cloud migration to AWS?

Answer:

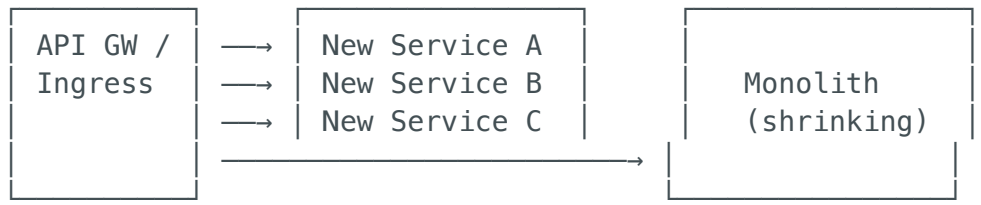
```
AWS CLOUD MIGRATION FRAMEWORK:

ASSESSMENT (Week 1-3):
├── AWS Migration Evaluator: Cost modeling (on-prem vs AWS)
├── AWS Application Discovery Service: Inventory + dependencies
├── Categorize each app (6 R's):
│   ├── Rehost (lift-and-shift): VM → EC2
│   ├── Replatform: VM → containers, DB → RDS
│   ├── Refactor: Monolith → microservices on EKS
│   ├── Repurchase: COTS → SaaS (e.g., CRM → Salesforce)
│   ├── Retire: Decommission unused apps
│   └── Retain: Keep on-prem (regulatory, latency)
└── Migration wave plan (dependency-ordered)

FOUNDATION (Week 4-6):
├── Landing zone: Organizations + Control Tower + Terraform
├── Networking: VPC, Transit Gateway, Direct Connect
├── Security: IAM, KMS, CloudTrail, GuardDuty
├── CI/CD: Jenkins/GitHub Actions + ArgoCD + ECR
└── Monitoring: Prometheus + Grafana + CloudWatch

MIGRATION (Wave-based):
├── Wave 1: Dev/test environments (low risk, validate process)
├── Wave 2: Stateless web apps (EC2 → EKS containers)
├── Wave 3: Databases (AWS DMS → Aurora, minimal downtime)
├── Wave 4: Mission-critical apps (with DR + rollback plan)
└── Each wave: Migrate → Test → Validate → Cutover → Monitor

OPTIMIZATION (Ongoing):
├── Right-size (Compute Optimizer)
├── Savings Plans / Reserved Instances
├── Containerize remaining EC2 workloads
├── Implement auto-scaling (Karpenter, ASG)
└── Adopt managed services (RDS, ElastiCache, OpenSearch)
```

Q26: How do you modernize a monolith to microservices on EKS?

Answer:

```
STRANGLER FIG PATTERN (Incremental Migration):

Phase 1: CONTAINERIZE THE MONOLITH
├── Package existing app as Docker container
├── Deploy on EKS (single container, same architecture)
├── Set up monitoring, logging, CI/CD
└── No code changes — just infrastructure modernization
```

```
Phase 2: EXTRACT SERVICES (one at a time)
├── Identify bounded contexts (DDD)
├── Extract highest-value service first
├── Route via API Gateway:
│   ├── New endpoint → microservice
│   └── Everything else → monolith
├── Database: Start with shared DB, migrate to DB-per-service later

Phase 3: EVOLVE
├── Add event-driven communication (SQS/SNS/EventBridge)
├── Implement service mesh (Istio) for mTLS
├── Each service gets: own CI/CD, own database, own team
└── Monolith shrinks to nothing
```



---

10. Scripting & Automation (Python, Bash, Go)

Q27: Show examples of automation scripts for AWS operations.

Answer:

```python
#!/usr/bin/env python3
"""
AWS Idle Resource Finder — Lambda-scheduled, runs nightly.
Identifies underutilized EC2, idle EBS, unattached EIPs.
"""
import boto3
from datetime import datetime, timedelta

ec2 = boto3.client('ec2')
cw = boto3.client('cloudwatch')

def find_idle_ec2():
    """EC2 instances with < 5% avg CPU over 7 days."""
    instances = ec2.describe_instances(
        Filters=[{'Name': 'instance-state-name', 'Values': ['running']}]
    )['Reservations']
```

```
    idle = []
    for res in instances:
        for inst in res['Instances']:
            iid = inst['InstanceId']
            metrics = cw.get_metric_statistics(
                Namespace='AWS/EC2', MetricName='CPUUtilization',
                Dimensions=[{'Name': 'InstanceId', 'Value': iid}],
                StartTime=datetime.utcnow() - timedelta(days=7),
                EndTime=datetime.utcnow(), Period=86400,
                Statistics=['Average']
            )
            if metrics['Datapoints']:
                avg = sum(d['Average'] for d in metrics['Datapoints']) /
len(metrics['Datapoints'])
                if avg < 5.0:
                    tags = {t['Key']: t['Value'] for t in inst.get('Tags',
[])}
                    idle.append({'id': iid, 'cpu': round(avg, 2),
                                 'type': inst['InstanceType'], 'name':
tags.get('Name', '')})
    return idle

def find_unattached_ebs():
    """EBS volumes in 'available' state (not attached)."""
    volumes = ec2.describe_volumes(Filters=[{'Name': 'status', 'Values':
['available']}])
    return [{'id': v['VolumeId'], 'size_gb': v['Size'], 'type':
v['VolumeType']}
            for v in volumes['Volumes']]
```

```
#!/bin/bash
# EKS namespace health check — CronJob every 5 minutes
set -euo pipefail

for NS in $(kubectl get ns -o jsonpath='{.items[*].metadata.name}'); do
  # Find unhealthy pods
  UNHEALTHY=$(kubectl get pods -n "$NS" \
    --field-selector=status.phase!=Running,status.phase!=Succeeded \
    -o jsonpath='{.items[*].metadata.name}' 2>/dev/null)

  if [[ -n "$UNHEALTHY" ]]; then
    echo "⚠️  [$NS] Unhealthy pods: $UNHEALTHY"
    # Post to Slack
    curl -sS -X POST "$SLACK_WEBHOOK" \
      -H "Content-Type: application/json" \
      -d "{\"text\":\"🔴 *Unhealthy pods in $NS*: \`$UNHEALTHY\`\"}"
  fi
done

# Check HPA at max replicas (scale-out exhausted)
kubectl get hpa -A -o json | jq -r '.items[] |
  select(.status.currentReplicas == .spec.maxReplicas) |
  "⚠️  HPA maxed: \(.metadata.namespace)/\(.metadata.name) at \
(.spec.maxReplicas) replicas"'
```

---

11. Configuration Management — Ansible

Q28: How do you use Ansible alongside Terraform and Kubernetes?

Answer:

```
TOOL RESPONSIBILITIES:

Terraform → PROVISION infrastructure (VPC, EKS, RDS, S3)
Ansible   → CONFIGURE what runs ON infrastructure (OS, packages, agents)
ArgoCD    → DEPLOY applications TO Kubernetes

Ansible Use Cases in a K8s-First World:
├── EC2 bastion/jump box hardening (CIS benchmark)
├── Self-managed database server configuration
├── CloudWatch agent installation on EC2
├── OS patching playbooks (scheduled maintenance)
```

```
├── Disaster recovery runbooks (automated failover steps)
└── Legacy VM configuration (during migration to containers)
```

```yaml
# Ansible Playbook: Harden EC2 instances for Mareana
---
- name: CIS-hardened base configuration
  hosts: all
  become: true

  tasks:
    - name: Ensure latest security patches
      yum:
        name: "*"
        state: latest
        security: yes

    - name: Disable unused services
      systemd:
        name: "{{ item }}"
        state: stopped
        enabled: no
      loop: [rpcbind, cups, avahi-daemon]
      ignore_errors: yes

    - name: Configure SSH hardening
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "{{ item.regexp }}"
        line: "{{ item.line }}"
      loop:
        - { regexp: '^#?PermitRootLogin', line: 'PermitRootLogin no' }
        - { regexp: '^#?PasswordAuthentication', line:
'PasswordAuthentication no' }
        - { regexp: '^#?X11Forwarding', line: 'X11Forwarding no' }
        - { regexp: '^#?MaxAuthTries', line: 'MaxAuthTries 3' }
      notify: Restart SSH

    - name: Install CloudWatch agent
      yum:
        name: amazon-cloudwatch-agent
        state: present

    - name: Deploy CloudWatch agent config
      template:
        src: cloudwatch-agent-config.json.j2
        dest: /opt/aws/amazon-cloudwatch-agent/etc/config.json
      notify: Restart CloudWatch Agent

  handlers:
    - name: Restart SSH
      systemd: { name: sshd, state: restarted }
    - name: Restart CloudWatch Agent
      systemd: { name: amazon-cloudwatch-agent, state: restarted }
```

12. Incident Management, RCA & Reliability

Q29: Describe your incident management and post-mortem process.

Answer:

```
INCIDENT MANAGEMENT LIFECYCLE:

DETECT → TRIAGE → CONTAIN → RESOLVE → POST-MORTEM

DETECTION (automated):
├── Prometheus alerts → AlertManager → PagerDuty (SEV1/2)
├── CloudWatch alarms → SNS → Slack (SEV3/4)
├── GuardDuty findings → Security Hub → Lambda → Slack
└── Synthetic monitoring (Route 53 health checks)

TRIAGE (< 5 minutes):
├── On-call acknowledges alert in PagerDuty
├── Assess severity: SEV1 (down), SEV2 (degraded), SEV3 (minor)
├── SEV1/2: Open Slack incident channel #inc-YYYY-MM-DD-description
├── SEV1: Page incident commander + engineering leads
└── Start documenting timeline

CONTAINMENT (< 30 minutes):
├── Rollback if deployment-related (ArgoCD one-click)
├── Scale up if capacity-related (Karpenter responds automatically)
├── Failover if AZ/region issue (Route 53 failover)
├── Block bad actors if security issue (WAF rules)
└── Communicate: Slack update, StatusPage if customer-facing

RESOLUTION:
├── Root cause identified and fix applied
├── Monitoring confirms recovery
├── Final communication to stakeholders
└── Incident channel archived

POST-MORTEM (within 48 hours):
├── Blameless (focus on systems, not people)
├── Timeline of events (minute-by-minute)
├── Root cause analysis (5 Whys technique)
├── Contributing factors
├── Action items: Preventive + Detective + Corrective
└── Review in weekly reliability meeting
```

Post-mortem example:

```
Title: [SEV2] API latency spike — 2024-06-15 (75 min)
Impact: p99 latency > 5s for 100% of API endpoints
Root cause: Aurora PostgreSQL connection pool exhaustion
5 Whys:
  1. Why high latency? → DB queries timing out
  2. Why timeouts? → All 200 connections in use
  3. Why connections exhausted? → Connection leak in new code path
  4. Why not caught? → No integration tests for connection lifecycle
  5. Why no alert? → Connection pool metric not in CloudWatch dashboard

Action items:
  [P1] Add db.connection_pool.active CloudWatch alarm (due: Jun 20)
  [P1] Fix connection leak in order-service (PR #1234, done)
  [P2] Add integration test for connection lifecycle (due: Jun 25)
  [P3] Add connection pool dashboard to Grafana (done)
```

---

13. Linux Administration & Troubleshooting

Q30: What Linux troubleshooting skills are critical for this role?

Answer:

Category	Commands	When to Use
Process	`top`, `htop`, `ps aux --sort=-pcpu`, `strace -p PID`	High CPU, hung processes, syscall debugging
Memory	`free -h`, `vmstat 1`, `cat /proc/meminfo`, `sar -r`	OOM investigation, swap usage, memory leak
Disk	`df -h`, `du -sh /var/*`, `iostat -x 1`, `lsblk`, `iotop`	Disk full, I/O bottleneck, slow writes
Network	`ss -tuln`, `netstat -anp`, `tcpdump -i eth0`, `curl -v`, `dig`, `traceroute`	Port conflicts, packet capture, DNS, connectivity

Category	Commands	When to Use
Logs	`journalctl -u service`, `dmesg -T`, `tail -f /var/log/`, `grep -r "ERROR"`	Error investigation, kernel messages
Containers	`docker stats`, `docker inspect`, `crictl ps`, `crictl logs`	Container resource usage, runtime debugging
K8s nodes	`kubectl describe node`, `kubectl top node`, `systemctl status kubelet`	Node issues, kubelet failures

Real troubleshooting scenario:

```
# Symptom: Pods OOMKilled on specific EKS node

# 1. Check node-level memory
kubectl describe node ip-10-0-11-45
# Look for: MemoryPressure=True, Allocatable memory

# 2. SSH to node (via SSM)
aws ssm start-session --target i-0abc123

# 3. Check what's consuming memory
free -h                          # Total/used/available
ps aux --sort=-rss | head -20    # Top memory consumers
dmesg -T | grep -i oom           # Kernel OOM killer logs
cat /proc/meminfo | grep -E "MemTotal|MemAvailable|SwapTotal"

# 4. Check container runtime
crictl stats                     # Per-container CPU/memory
crictl inspect <container-id>    # Container resource limits

# 5. Root cause: kube-proxy in iptables mode consuming 2GB
# Fix: Switch to IPVS mode or reduce service count
```

---

14. AWS Cost Optimization & FinOps

Q31: How do you implement cost optimization on AWS?

Answer:

Strategy	Savings	Implementation
Spot via Karpenter	60-70%	EKS node pools with `spot` capacity type
Graviton (ARM)	20%	`arm64` constraint in Karpenter NodePool
Savings Plans	Up to 72%	1-year compute Savings Plans for steady-state
RDS Reserved	40-60%	1-year partial-upfront for production databases
S3 Lifecycle	30-50%	Standard → IA → Glacier tiering
VPC Endpoints	Variable	Avoid NAT Gateway data charges ($0.045/GB)
EBS gp3 vs gp2	20%	gp3 is cheaper and faster by default
Auto-shutdown	60%+	Lambda stops non-prod resources at 7 PM, starts at 9 AM
Right-sizing	20-40%	Compute Optimizer recommendations, Kubecost

Cost governance framework:

```
VISIBILITY:
├── Cost Explorer: Daily spend by service
├── Kubecost: Per-namespace, per-deployment cost in K8s
├── Infracost: Cost estimation in Terraform PRs
├── AWS Budgets: Alerts at 80% and 100%
└── Mandatory tags: Project, Team, Environment, CostCenter

OPTIMIZATION:
├── Weekly: Review Cost Explorer anomalies
├── Monthly: Right-sizing review (Compute Optimizer)
├── Quarterly: Savings Plans / RI analysis
└── Per-PR: Infracost diff in pull request comments

GOVERNANCE:
├── SCPs: Block expensive instance types in sandbox
├── Auto-expire sandbox resources (EventBridge + Lambda)
├── Budget actions: Auto-stop non-prod at 100% budget
└── Unit economics tracking: cost-per-customer, cost-per-API-call
```

---

15. Behavioral & Leadership

Q32: Tell me about yourself.

Answer:
"I'm a Cloud & DevOps Architect with 12+ years of experience, primarily on AWS, building enterprise-grade infrastructure platforms.

Highlights most relevant to this role:

1. **AWS Architecture**: Designed multi-account architectures using Organizations + Control Tower, with Transit Gateway networking, private EKS clusters, and full DevSecOps integration.

2. **Kubernetes (EKS)**: Managed production EKS clusters serving 100+ microservices. Implemented Karpenter (35% cost reduction), hardened security per CIS benchmarks, and built zero-downtime upgrade processes.

3. **CI/CD & GitOps**: Built end-to-end pipelines with Jenkins + ArgoCD — from code commit to production deployment with security scanning (Trivy, SonarQube, tfsec) at every stage.

4. **Terraform**: 100% IaC-managed infrastructure. Built reusable Terraform modules used across 10+ projects, with state management, drift detection, and cost estimation in PR reviews.

5. **Team Leadership**: Led an 8-person cloud engineering team. Established on-call rotations, blameless post-mortem culture, and quarterly DR drills.

I'm excited about Mareana because it's a product company building AI/ML solutions for manufacturing — the infrastructure challenges are genuinely interesting (IoT data streams, ML inference at scale, multi-tenant SaaS), and the Cloud Architect role directly enables the product's reliability and scalability."

Q33: Describe a complex technical problem you solved.

Answer:
"Our production EKS cluster experienced random 5xx errors during peak hours — affecting 15% of API requests, but only between 9 AM and 6 PM.

Investigation:

1. Prometheus showed no resource pressure (CPU/memory fine on pods)
2. ALB target health checks passing, but response time spiking
3. Deeper: `kubectl top nodes` showed nodes near memory capacity
4. Root cause: kube-proxy in iptables mode — with 200+ K8s Services, iptables rule chains consumed 2GB RAM per node, leaving insufficient memory for pods during peak

Solution:

1. Migrated kube-proxy from iptables to IPVS mode (O(1) lookup vs O(n))
2. Node memory overhead dropped from 2GB to 200MB
3. Added `--system-reserved=memory=1Gi` to kubelet configuration
4. Implemented VPA for right-sizing pod resource requests

Impact: 5xx errors dropped from 15% to 0.01%, node memory freed up 20%, and we avoided a $3K/month cluster expansion.

Post-mortem action items:

- Added node-level memory monitoring in Grafana
- Documented kube-proxy mode as architecture decision record
- Added iptables rule count alert for early detection"

Q34: How do you lead and mentor a DevOps team?

Answer:
"My leadership philosophy:

1. **Architecture Decision Records (ADRs)**: Every significant decision documented — what we chose, why, and what we rejected. Prevents re-litigation and builds shared knowledge.

2. **On-call culture**: Fair rotation, clear escalation, and I participate in on-call personally. Leaders should not exempt themselves from production responsibility.

3. **PR reviews as teaching**: I write detailed, educational reviews — explaining *why* certain patterns are preferred, linking to documentation, sharing relevant war stories.

4. **Automation KPI**: Track 'toil hours' monthly. If someone does the same manual task 3 times, we automate it. Target: < 20% time spent on toil.

5. **Weekly tech deep-dives**: 30-minute team sessions — AWS re:Invent recaps, new Terraform features, debugging techniques, security advisories.

6. **Career growth**: Monthly 1:1s focused on career trajectory (architecture, SRE, management). Support certification goals (AWS SA Pro, CKA) with study groups and exam budget."

Q35: How do you handle architecture disagreements?

Answer:
"Data-driven decision-making:

1. **Listen first**: The team has operational context I may lack. Their concern about complexity or maintenance burden is usually valid.

2. **Comparison matrix**: Concrete criteria — performance benchmarks, cost, operational complexity, team skills, time-to-implement.

3. **Time-boxed PoC**: For fundamental disagreements, 1-week proof-of-concept with measurable success criteria defined upfront. Example: Team wanted Nginx Ingress, I suggested Istio. We PoC'd both — Nginx won on simplicity for the current scale.

4. **ADR**: Document the decision, rationale, alternatives considered, and accepted trade-offs. This respects the 'losing' argument and provides context for future team members.

5. **Commit and move on**: Once decided, everyone commits fully. No 'I told you so' if issues arise later — we iterate and improve."

---

16. Mareana-Specific Questions

Q36: Why Mareana?

Answer:
"Three reasons:

1. **Product company**: Mareana builds its own products — I'd own the platform end-to-end rather than rebuilding for different clients. The feedback loop between infrastructure quality and product quality is direct.

2. **Domain**: Manufacturing, sustainability, and supply chain are data-intensive — IoT sensor streams, time-series analytics, ML inference at scale. These create genuinely interesting infrastructure challenges: edge-to-cloud data pipelines, multi-tenant isolation, low-latency inference.

3. **Platform impact**: As the Cloud Architect, I'd build the foundation that all product engineering runs on. Getting the platform right — reliability, security, cost efficiency, developer experience — is a force multiplier for the entire company."

Q37: How would you design the cloud platform for Mareana's AI/ML products?

Answer:

```
MAREANA PLATFORM ARCHITECTURE:

EDGE (Factory Floor):
├── IoT sensors → AWS IoT Core → IoT Rule → Kinesis
├── Edge compute: Greengrass (local inference, data filtering)
└── MQTT/HTTPS → TLS encrypted

INGESTION:
├── Kinesis Data Streams (real-time sensor telemetry)
├── Kinesis Firehose → S3 (raw data archive, Parquet)
├── API Gateway (REST — batch uploads, partner APIs)
└── EventBridge (internal event routing)

COMPUTE (EKS):
├── Application services (microservices)
```

```
├── ML inference endpoints (SageMaker / custom on EKS GPU nodes)
├── Data processing (Spark on EMR / EKS)
├── Auto-scaling: Karpenter (CPU), KEDA (event-driven from SQS)
└── GPU pool: g5.xlarge (ML inference, Spot for batch training)

DATA:
├── S3 Data Lake (Bronze/Silver/Gold medallion architecture)
├── Aurora PostgreSQL (transactional — multi-AZ, encrypted)
├── DynamoDB (IoT metadata, session state, high-throughput)
├── ElastiCache Redis (API cache, ML feature store)
├── Timestream (time-series sensor data — purpose-built)
└── Redshift Serverless (analytics, BI dashboards)

SECURITY & GOVERNANCE:
├── Organizations + Control Tower (multi-account)
├── IAM Identity Center (SSO, federated)
├── KMS CMK (all data encrypted at rest)
├── GuardDuty + Security Hub (threat detection)
├── Private EKS (VPC endpoints for all AWS services)
└── Terraform + tfsec + ArgoCD + OPA (IaC + GitOps + policy)

OBSERVABILITY:
├── Prometheus + Grafana (K8s metrics)
├── Fluent Bit → CloudWatch Logs / OpenSearch
├── AWS X-Ray (distributed tracing)
├── Kubecost (cost attribution)
└── Route 53 health checks (uptime monitoring)
```

Q38: How would you handle multi-tenancy for Mareana's SaaS platform?

Answer:

Strategy	Isolation Level	Best For	Trade-off
Namespace-per-tenant	Medium (logical)	Small/medium tenants	Shared infra cost ↓, noisy neighbor risk ↑
Cluster-per-tenant	High (physical)	Large enterprise clients	Strong isolation, higher cost
Account-per-tenant	Highest (full AWS)	Regulated / compliance-heavy	Maximum isolation, highest management cost

My recommendation for Mareana:

- **Default**: Namespace-per-tenant (K8s NetworkPolicies + ResourceQuotas for isolation)
- **Premium tier**: Dedicated cluster for large manufacturing clients
- **Data**: Schema-per-tenant in Aurora (balance isolation vs. management)
- **Cost attribution**: Kubecost labels per tenant namespace

---

17. Azure & GCP — Quick Reference

> *Note: This section provides comparison knowledge — the role is AWS-primary.*

Q39: How do key AWS services map to Azure and GCP equivalents?

Answer:

Category	AWS	Azure	GCP
Compute	EC2	Virtual Machines	Compute Engine
Kubernetes	EKS	AKS (free control plane)	GKE (most advanced, Autopilot)
Serverless	Lambda	Azure Functions	Cloud Functions
Object Storage	S3	Blob Storage	Cloud Storage
Relational DB	RDS / Aurora	Azure SQL / Cosmos DB (Postgres)	Cloud SQL / AlloyDB
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable
Cache	ElastiCache	Azure Cache for Redis	Memorystore
Container Registry	ECR	ACR	Artifact Registry
IaC	CloudFormation	ARM / Bicep	Deployment Manager
CI/CD	CodePipeline	Azure DevOps	Cloud Build

Category	AWS	Azure	GCP
IAM	IAM + Identity Center	Entra ID + RBAC	Cloud IAM
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring
VPN	Site-to-Site VPN	VPN Gateway	Cloud VPN
Private Link	PrivateLink	Private Endpoint	Private Service Connect
DNS	Route 53	Azure DNS	Cloud DNS
WAF	AWS WAF	Azure Front Door WAF	Cloud Armor
Secrets	Secrets Manager	Key Vault	Secret Manager
Data Warehouse	Redshift	Synapse Analytics	BigQuery

Q40: When would you recommend Azure or GCP over AWS?

Answer:

Scenario	Recommendation	Why
Client uses Microsoft 365 / Entra ID heavily	**Azure**	Seamless SSO, Entra ID integration
Data analytics is the primary workload	**GCP**	BigQuery is unmatched for analytics
Client needs cheapest Kubernetes	**AKS**	Free control plane, easiest setup
Client needs best Kubernetes features	**GKE**	Autopilot, most advanced K8s
Client wants broadest service catalog	**AWS**	Most services, most regions, most certifications

Scenario	Recommendation	Why
Client is in manufacturing (Mareana's domain)	**AWS**	IoT Core, Greengrass, broadest ML services
Multi-cloud / portability required	**All three**	Terraform + K8s abstract the differences

JD Coverage Checklist

JD Requirement	Question(s)	Status
AWS architecture & services (EC2, S3, RDS, IAM, VPC, Lambda)	Q1-Q6	✅
Kubernetes at scale (EKS, security, lifecycle)	Q11-Q15	✅
CI/CD (Jenkins, GitHub Actions, GitLab CI/CD)	Q16-Q18	✅
GitOps (ArgoCD / FluxCD)	Q17	✅
Terraform / CloudFormation / Pulumi	Q19-Q20	✅
Docker & container security	Q21	✅
VPC, Transit Gateway, Security Groups, NACLs, PrivateLink	Q7-Q10	✅
Monitoring (Prometheus, Grafana, ELK, CloudWatch)	Q22	✅
Cloud security, IAM, DevSecOps	Q23-Q24	✅
Cloud migration & modernization	Q25-Q26	✅
Scripting (Python, Bash)	Q27	✅
Configuration management (Ansible)	Q28	✅
Incident management, RCA, post-mortem	Q29	✅
Linux/Windows servers, IT operations	Q30	✅
Technical leadership & mentoring	Q32-Q35	✅
Cost optimization / FinOps	Q31	✅
Hybrid/multi-cloud knowledge	Q8, Q39-Q40	✅

JD Requirement	Question(s)	Status
Networking, DNS, VPN, firewalls	Q7-Q10	✅
EventBridge, SNS/SQS	Q4, Q37	✅
GuardDuty, Security Hub, CloudTrail, Config	Q23-Q24	✅
Stakeholder communication	Q32-Q35	✅

Interview Tips for Mareana

1. **Lead with AWS depth** — EKS, VPC, IAM, S3 — show deep, hands-on knowledge
2. **Show Kubernetes mastery** — Karpenter, IRSA, security hardening, upgrades — this is a core skill
3. **Emphasize automation** — Terraform, CI/CD, GitOps. "If it's not code, it doesn't exist"
4. **Talk reliability** — Incident management, post-mortems, DR — production maturity matters
5. **Manufacturing context** — IoT, sensor data, time-series, edge computing — show domain awareness
6. **Cost consciousness** — Product company margins matter. Show Spot, Graviton, FinOps awareness
7. **Security is non-negotiable** — DevSecOps, least-privilege, encryption — weave security into every answer
8. **Leadership** — They want a leader who can build and guide a team, not just architect in isolation

---

Prepared: June 2026 | **Candidate:** Pushparaj Naik | **Role:** Cloud & DevOps Architect — Mareana